ACADEMIC PROJECT TECHNICAL DOSSIER

SMART PATIENT HEALTH MONITORING SYSTEM

IoT-Based Real-Time In-Patient Vital Signs Telemetry & Central Hospital Dashboard

A Cyber-Physical Clinical Healthcare Telemetry Solution leveraging ESP32 IoT Edge Nodes, Optical Photoplethysmography (Heart Rate & SpO■), Dallas 1-Wire Digital Thermometry, Node.js/FastAPI REST & WebSocket Gateways, PostgreSQL Relational Storage, and a React Live Hospital Dashboard.

Course / Degree:	Master of Computer Applications (MCA)
Core Domains:	Internet of Things (IoT), Full-Stack Engineering, Biomedical Informatics
Application Area:	General Inpatients, Intensive Care Units (ICU), Post-Operative Recovery
Hardware IoT Node:	ESP32 Dev Module (Node ID: ESP32-001)
Primary Sensors:	MAX30102 (Pulse Rate & SpO■), DS18B20 (Body Temperature)
Backend Server:	Node.js Express / Python FastAPI REST & WebSocket Gateway (Port 5000)
Database:	PostgreSQL 16 Relational Engine (Health Monitoring Database)
Frontend Dashboard:	React 18 + Vite + TailwindCSS SPA (Port 3000)
Containerization:	Docker & Docker Compose (3-Tier Microservices Architecture)
Documentation Date:	October 2026

1. Executive Summary & Clinical Objectives

The **Smart Patient Health Monitoring System** is an end-to-end cyber-physical IoT telemetry solution engineered for continuous, automated observation of vital physiological signs for hospitalized patients across general inpatient wards, recovery suites, and intensive care units. It is an open, universal vital signs monitoring platform, not a single-disease diagnostic tool.

Clinical Problem Solved: In conventional hospital wards, vital signs are manually collected by nursing staff every 4 to 8 hours. Between these observation cycles, sudden physiological deterioration—such as silent hypoxia, cardiac arrhythmias, or sudden febrile spikes—can go unnoticed. This platform automates continuous vital collection, streaming data wirelessly every **3 seconds** directly from the patient's bedside to a central hospital dashboard, triggering immediate alarms if vitals cross safe clinical boundaries.

Key Innovation — Resilient Clinical Fault Tolerance: Unlike naive IoT prototypes where sensor disconnection or wire faults cause firmware crashes, this system implements automated fault-tolerant sensing. If the physical I2C sensor disconnects or encounters bus failure, the ESP32 firmware dynamically engages its onboard clinical simulation algorithm to maintain unbroken telemetry streaming, while the dashboard clearly informs clinical staff of the hardware disconnection.

2. System Architecture & 5-Layer Model

Layer	Primary Technologies	Key Functions & Responsibilities
1. Perception Layer	ESP32, MAX30102, DS18B20, Active Buzzer, Status LED	50 Hz optical PPG wave sampling, temperature analog-to-digital conversion, local audio alarm triggering.
2. Network Layer	Wi-Fi 802.11 b/g/n, HTTP/1.1 REST, WebSockets	Secure JSON packaging, transmission every 3 seconds, non-blocking auto-reconnect on network dropouts.
3. Processing Layer	Node.js Express, Python FastAPI, Uvicorn	Telemetry ingestion, mathematical sanitization, clinical rule boundary evaluation, WebSocket broadcast.
4. Database Layer	PostgreSQL 16 Relational Engine	Time-series telemetry persistence, patient demographic registry, clinical alert tracking, IoT device registry.
5. Presentation Layer	React 18, Vite, TailwindCSS, Recharts	Live clinical gauges, real-time hardware status beacons, multi-bed ward registry, interactive trend curves.

3. Hardware Specifications & Pin Configuration

Component	Model / Spec	Operating Voltage	ESP32 Wiring / Protocol
Microcontroller	ESP-WROOM-32 (Dual-Core 240MHz, 520KB SRAM)	5V USB / 3.3V Logic	Main edge computing node & Wi-Fi station
Pulse Oximeter	MAX30102 (Red 660nm + IR 880nm Optical PPG)	3.3V - 5V	SDA → GPIO 21, SCL → GPIO 22 (I2C at 100 kHz)
Temperature Sensor	DS18B20 Digital Thermometer (±0.5°C accuracy)	3.0V - 5.5V	DATA → GPIO 4 (1-Wire with 4.7kΩ pull-up)
Local Alarm Buzzer	5V Active Piezoelectric Buzzer	3.3V - 5V	POS → GPIO 18, NEG → GND
Status Indicator	Onboard SMD Blue LED	3.3V	GPIO 2 (Blinks on telemetry transmission)

4. PostgreSQL Relational Database Schema

Database Engine: **PostgreSQL 16**. The relational schema enforces strict foreign key constraints, timestamp indexing, and ACID guarantees:

Table Name	Primary / Foreign Keys	Stored Attributes & Data Types
patients	patient_id (VARCHAR, PK)	name, age, gender, room_number, admission_date, status, comorbidities, contact_emergency
telemetry	id (BIGSERIAL, PK) patient_id (FK)	heart_rate (NUMERIC), spo2 (NUMERIC), temperature (NUMERIC), respiration_rate, battery_level, device_id, timestamp
alerts	id (BIGSERIAL, PK) patient_id (FK)	alert_type, severity, value, threshold, message, resolved (BOOLEAN), timestamp
devices	device_id (VARCHAR, PK) patient_id (FK)	status ('ONLINE'/'OFFLINE'), ip_address, last_seen (TIMESTAMPTZ), firmware_version
health_readings	id (BIGSERIAL, PK) patient_id (FK)	Historical time-series archive for multi-parameter clinical charting

5. Clinical Thresholds & Medical Alarm Rules

Vital Sign	Normal Range	Warning Boundary	Critical Alarm	Automated System Action
Blood Oxygen (SpO■)	95.0% - 100.0%	92.0% - 94.9% (Mild Hypoxia)	< 92.0% (Severe Hypoxia)	Buzzer ON (GPIO 18), DB alert logged, UI flashes red
Body Temperature	36.10°C - 37.20°C	37.30°C - 38.00°C (Low Fever)	> 38.00°C (Pyrexia)	Bedside alarm active, Temperature card warning state
Heart Rate (Pulse)	60 - 100 BPM	50 - 59 BPM (Bradycardia)	> 100 BPM (Tachycardia)	Logged into Clinical Alerts registry with timestamp

6. Containerized Deployment (Docker Microservices)

The platform is packaged into 3 isolated Docker microservices orchestrated via docker-compose.yml:

Container Service	Base Image	Port Mapping	Service Description
postgres_db	postgres:16-alpine	5432 : 5432	Persistent database with automated health checks and named data volume.
health_monitor_api	python:3.12-slim / node:20	5000 : 5000	Telemetry ingestion gateway, REST endpoints, and WebSocket push server.
health_monitor_dashboard	nginx:1.25-alpine	80 : 3000	Production build of React Single Page Application served via Nginx reverse proxy.

7. Testing & Demonstration Guide

Demonstration Procedure for Project Evaluation:

Step 1: Physical Stream Verification: Open Arduino Serial Monitor at 115200 baud. Observe live pulse rate, SpO■, and body temperature printed every 1 second, and HTTP POST (Code 201) transmission every 3 seconds.

Step 2: Database Audit: Query PostgreSQL to verify real incoming telemetry rows with millisecond timestamps and increasing ID sequence.

Step 3: Real-Time Dashboard: Open http://localhost:3000 to observe the ■ **LIVE ESP32 HARDWARE STREAM ACTIVE** banner with dynamic gauges and trend charts.

Step 4: Live Hardware Disconnect Test: Unplug the ESP32 USB cable. Within 15 seconds, the dashboard turns ■ **RED: [!] ESP32 HARDWARE NOT CONNECTED / OFFLINE**. Plug back in, and it recovers to ■ **GREEN** in 3 seconds.

8. Project Viva Examination Q&A; Cheat Sheet

Examiner Question	Recommended Academic Response
What is the clinical scope of this project?	It is a general in-patient vital signs monitoring and telemetry system designed for continuous tracking of Heart Rate, SpO■, and Temperature across hospital wards and recovery units. It is an open physiological monitoring platform, not a single-disease diagnostic tool.
Why ESP32 instead of Arduino Uno or Raspberry Pi?	Arduino Uno lacks built-in networking. Raspberry Pi consumes high power and is expensive for edge nodes. The ESP32 offers dual-core 240MHz processing, hardware I2C/1-Wire controllers, built-in 2.4GHz Wi-Fi, and low power sleep modes at minimal cost.
How does the MAX30102 calculate SpO■?	It emits Red (660nm) and Infrared (880nm) light into fingertip capillaries. Deoxygenated hemoglobin absorbs more Red light, while oxygenated hemoglobin absorbs more Infrared light. The ratio of ratios R = (AC_red / DC_red) / (AC_ir / DC_ir) maps to calibration curves to calculate SpO■ percentage.
How does the system ensure patient privacy?	The system uses pseudonymized identifiers (P001, PAT-101) rather than transmitting Personally Identifiable Information (PII) like national identity numbers or names in the telemetry packets.
What happens if a sensor wire disconnects?	The firmware implements **resilient fault tolerance**: if the I2C bus encounters a hardware fault, the onboard clinical model maintains vital sign continuity to prevent firmware crash, while the dashboard informs clinical staff of the hardware disconnection.
Why choose PostgreSQL over NoSQL (MongoDB)?	Clinical health monitoring requires relational integrity (patients are strictly linked to devices, rooms, and alerts). PostgreSQL provides ACID transactional guarantees, strong data typing, and native support for time-series indexing.